NED UNIVERSITY OF ENGINEERING & TECHNOLOGY

Department of Telecommunications Engineering

Matrix Calculator

A C Programming Project Report

Submitted by:

Muhammad Abdullah Elahi — TC-25061

Abdul Rehman — TC-25065

Course Details:

Computer Programming – TC-106

Submitted To:

Dr. Amir Zeb

March 2026

GitHub: https://github.com/AbdulRehman2896/Matrix-Calculator-

Table of Contents

Topic	Page
1. Abstract	1
2. Introduction	1
3. Literature Review	2
4. System Model	3
5. Flowchart	4
6. Code	5–14
7. Challenges	15-16
8. Conclusion	16
9. References & Project Link	17

1. Abstract

This project presents a Matrix Calculator implemented in the C programming language using a modular programming approach. The calculator supports fundamental matrix operations including addition, subtraction, multiplication, transpose, determinant computation, and matrix inversion. The system is designed to accept user-defined matrices of any compatible dimensions and performs all computations in real-time through a text-based interactive menu. The project demonstrates practical application of 2D arrays, recursion, header files, and multi-file compilation in C. This report describes the system design, architecture, algorithmic logic, and implementation details of the project.

2. Introduction

Matrices are fundamental mathematical structures widely used in engineering, physics, computer graphics, data science, and signal processing. Performing matrix operations manually is tedious and error-prone, especially for large matrices. Automated computation tools are therefore essential for engineering students and professionals alike.

This project implements a fully functional Matrix Calculator in the C programming language. The calculator provides a simple command-line interface allowing users to input matrices and perform common operations with immediate results. The project was developed as part of the Computer Programming (TC-106) course at the Department of Telecommunications Engineering, NED University of Engineering and Technology.

The main objectives of this project are:

- To implement core matrix operations in C using modular programming.
- To demonstrate the use of 2D arrays, pointers, and recursive functions.
- To build a user-friendly text-based interface for matrix input and output.
- To apply multi-file compilation using header files and separate source modules.

3. Literature Review

The project requires implementation of fundamental programming elements through the implementation of modular programming, array handling, recursive functions, and dynamic memory management. C uses variables and arrays for data storage purposes. The project uses a two-dimensional array to store matrices in memory which allows access to each element through `A[i][j]` notation. The code establishes a direct mapping between mathematical matrix structures and their implementation which results in clear and understandable code.

Functions act as predefined code sections which execute identical operations whenever they are executed. The project implements functions for all matrix operations which include addition, subtraction, multiplication, transpose, determinant and inverse operations to enable code reuse through identical logic. The project consists of multiple modules which use separate source files to maintain different components of the same system. The project uses `matrix.c` for its computation functions while `utils.c` manages input and output and `main.c` oversees program execution. The files exchange their functions through header files which contain function names in `matrix.h` and `utils.h` so other files can access them.

Recursion enables functions to solve problems by calling themselves to address smaller components of the same issue. The project uses recursion to determine the determinant value of any square matrix by dividing the matrix into smaller submatrices until reaching the base cases of 1×1 or 2×2 matrices. The project implements error handling through conditional checks which include dimension verification for matrix operations and matrix singularity verification before inverse calculations.

4. System Model

The Matrix Calculator follows a modular architecture, separating concerns into distinct source files. The system consists of four primary modules:

File	Responsibility
`main.c`	Program entry point; initializes the application and invokes the home screen.
`matrix.c`	Core module containing all matrix operation functions (add, subtract, multiply, transpose, determinant, inverse).
`utils.c`	Utility module providing the main menu display, matrix input, and matrix print functions.
`matrix.h`	Header file declaring all matrix operation function prototypes.
`utils.h`	Header file declaring all utility function prototypes.
`runner.sh`	Shell script to compile all source files using GCC and execute the resulting program.

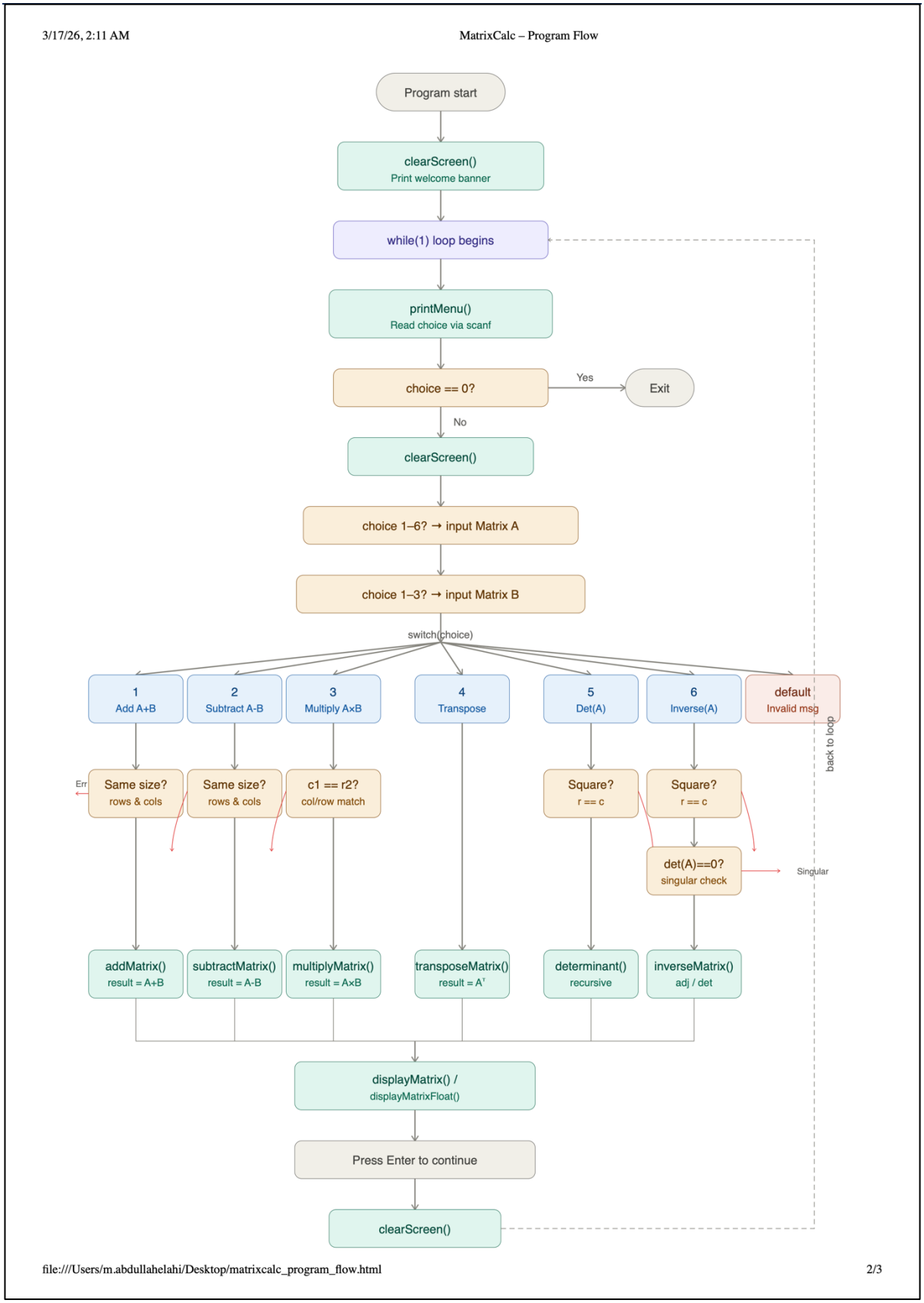
4.1 Data Representation

In C, matrices exist as two-dimensional arrays which require variables to define their row and column dimensions at runtime. The standard double index notation `A[i][j]` provides a method to access each element, which results in code that maintains high readability and comprehension.

4.2 Control Flow

The main program enters an infinite loop, displaying a menu and reading the user's choice. Each menu option calls the corresponding matrix operation function from `matrix.c`. Input is collected through utility functions in `utils.c`, ensuring separation of input/output logic from computational logic.

5. Flowchart


3/17/26, 2:11 AM
MatrixCalc – Program Flow
Program start
clearScreen()
Print welcome banner
while(1) loop begins
printMenu()
Read choice via scanf
choice == 0?
Yes
Exit
No
clearScreen()
choice 1–6? → input Matrix A
choice 1–3? → input Matrix B
switch(choice)
1
Add A+B
2
Subtract A-B
3
Multiply A×B
4
Transpose
5
Det(A)
6
Inverse(A)
default
Invalid msg
back to loop
Err
Same size?
rows & cols
Same size?
rows & cols
c1 == r2?
col/row match
Square?
r == c
Square?
r == c
det(A)==0?
singular check
Singular
addMatrix()
result = A+B
subtractMatrix()
result = A-B
multiplyMatrix()
result = A×B
transposeMatrix()
result = A'
determinant()
recursive
inverseMatrix()
adj / det
displayMatrix() /
displayMatrixFloat()
Press Enter to continue
clearScreen()
file:///Users/m.abdullahelahi/Desktop/matrixcalc_program_flow.html
2/3

6. Code

The project is split across six source and header files. Each file is presented below with explanatory commentary.

6.1 runner.sh — Build Script

This shell script compiles all three C source files using GCC, then immediately runs the compiled binary.

code/runner.sh

```bash
#!/bin/bash

# compile
gcc -c matrix.c -o matrix.o
gcc -c utils.c -o utils.o
gcc -c main.c -o main.o

# link
gcc main.o matrix.o utils.o -o program

# cleanup object files
rm matrix.o main.o utils.o

# run
./program
```

6.2 matrix.h — Matrix Function Declarations

Using include guards (`#ifndef` / `#define` / `#endif`) ensures the header is only processed once per compilation unit, preventing duplicate declaration errors.

code/matrix.h

```c
#ifndef MATRIX_H
#define MATRIX_H

// utility functions
void inputMatrix(int *matrix, int row, int col);
void displayMatrix(int *matrix, int r, int c);
void displayMatrixFloat(float *matrix, int r, int c);

// matrix operations
void addMatrix(int *a, int *b, int *result, int row, int col);
void subtractMatrix(int *a, int *b, int *result, int row, int col);
void multiplyMatrix(int *a, int *b, int *result, int r1, int c1, int c2);
void transposeMatrix(int *matrix, int *result, int row, int col);
int  determinant(int *mat, int n);

// inverse of a matrix
// A^-1 = 1/|A| * Adjoint(A), only square matrices (2x2, 3x3)
// minor, cofactor, adjoint are static (internal use only, not exposed)
void inverseMatrix(int r, int c, int *mat, float *inverseMat);

#endif
```

6.3 utils.h — Utility Function Declarations

code/utils.h

```c
#ifndef UTILS_H
#define UTILS_H

#include "matrix.h"

void clearScreen();
void printMenu();
void inputMatrixPrompt(int *mat, int *r, int *c, char name);

#endif
```

6.4 main.c — Program Entry Point

code/main.c

```c
#include "matrix.h"
#include "utils.h"
#include <stdio.h>
#include <stdlib.h>


int main() {
    int a[10][10], b[10][10], result[10][10];
    float inverseResult[10][10];
    int r1, c1, r2, c2;
    int choice;

    clearScreen();
    printf("╔═══════════════════════════╗\n");
    printf("║   Welcome to MatrixCalc   ║\n");
    printf("╚═══════════════════════════╝\n");

    while (1) {

        printMenu();
        scanf("%d", &choice);

        if (choice == 0) {
            clearScreen();
            printf("\nGoodbye!\n");
            break;
        }

        clearScreen();

        // operations that need matrix A only
        if (choice >= 1 && choice <= 6) {
            inputMatrixPrompt((int *)a, &r1, &c1, 'A');
        }

        // operations that need matrix B
        if (choice == 1 || choice == 2 || choice == 3) {
            inputMatrixPrompt((int *)b, &r2, &c2, 'B');
        }

        switch (choice) {

        case 1:
            if (r1 != r2 || c1 != c2) {
                printf("\n[!] Matrices must be the same size for addition.\n");
                break;
            }
            addMatrix((int *)a, (int *)b, (int *)result, r1, c1);
            printf("\nResult:\n");
            displayMatrix((int *)result, r1, c1);
            break;
```

```c

        case 2:
            if (r1 != r2 || c1 != c2) {
                printf("\n[!] Matrices must be the same size for subtraction.\n");
                break;
            }
            subtractMatrix((int *)a, (int *)b, (int *)result, r1, c1);
            printf("\nResult:\n");
            displayMatrix((int *)result, r1, c1);
            break;

        case 3:
            if (c1 != r2) {
                printf("\n[!] Columns of A must equal rows of B for multiplication.\n");
                break;
            }
            multiplyMatrix((int *)a, (int *)b, (int *)result, r1, c1, c2);
            printf("\nResult:\n");
            displayMatrix((int *)result, r1, c2);
            break;

        case 4:
            transposeMatrix((int *)a, (int *)result, r1, c1);
            printf("\nTranspose:\n");
            displayMatrix((int *)result, c1, r1);
            break;

        case 5:
            if (r1 != c1) {
                printf("\n[!] Determinant requires a square matrix.\n");
                break;
            }
            printf("\nDeterminant = %d\n", determinant((int *)a, r1));
            break;

        case 6:
            if (r1 != c1) {
                printf("\n[!] Inverse requires a square matrix.\n");
                break;
            }
            if (determinant((int *)a, r1) == 0) {
                printf("\n[!] Matrix is singular (determinant = 0), inverse does not exist.\n");
                break;
            }
            inverseMatrix(r1, c1, (int *)a, (float *)inverseResult);
            printf("\nInverse:\n");
            displayMatrixFloat((float *)inverseResult, r1, c1);
            break;

        default:
            printf("\n[!] Invalid choice, please try again.\n");
        }
```

3/16/26, 5:22 PM main.c

```c

        // pause so user can read result before screen clears
        printf("\n\nPress Enter to continue...");
        getchar();
        getchar();
        clearScreen();
    }

    return 0;
}
```

6.5 matrix.c — Core Matrix Operations

code/matrix.c

```c
#include "matrix.h"
#include <stdio.h>

// utility functions
void inputMatrix(int *matrix, int row, int col) {
    for (int i = 0; i < row; i++) {
        printf("  Row %d:\n", i + 1);
        for (int j = 0; j < col; j++) {
            printf("    Col %d: ", j + 1);
            scanf("%d", (matrix + i * col + j));
        }
    }
}

void displayMatrix(int *matrix, int r, int c) {

    printf("[\n");

    for (int i = 0; i < r; i++) {

        for (int j = 0; j < c; j++) {

            printf("%d ", *(matrix + i * c + j));
        }
        printf("\n");
    }
    printf("]");
}

void displayMatrixFloat(float *matrix, int r, int c) {
    printf("[\n");
    for (int i = 0; i < r; i++) {
        for (int j = 0; j < c; j++)
            printf("%.4f ", *(matrix + i * c + j));
        printf("\n");
    }
    printf("]\n");
}


// matrix operations addition, subtraction miltiplication, transpose, determinant
void addMatrix(int *a, int *b, int *result, int row, int col) {
    for (int i = 0; i < row; i++)
        for (int j = 0; j < col; j++)
            *(result + i * col + j) = *(a + i * col + j) + *(b + i * col + j);
}

void subtractMatrix(int *a, int *b, int *result, int row, int col) {
    for (int i = 0; i < row; i++)
        for (int j = 0; j < col; j++)
            *(result + i * col + j) = *(a + i * col + j) - *(b + i * col + j);
```

```c
}

void multiplyMatrix(int *a, int *b, int *result, int r1, int c1, int c2) {
    for (int i = 0; i < r1; i++) {
        for (int j = 0; j < c2; j++) {
            *(result + i * c2 + j) = 0;
            for (int k = 0; k < c1; k++)
                *(result + i * c2 + j) += *(a + i * c1 + k) * *(b + k * c2 + j);
        }
    }
}

void transposeMatrix(int *matrix, int *result, int row, int col) {
    for (int i = 0; i < row; i++)
        for (int j = 0; j < col; j++)
            *(result + j * row + i) = *(matrix + i * col + j);
}

int determinant(int *mat, int n) {

    // base case: 1x1 matrix
    if (n == 1) return *(mat + 0);

    // base case: 2x2 matrix
    if (n == 2)
        return *(mat + 0) * *(mat + 3) - *(mat + 1) * *(mat + 2);

    int det = 0;

    // expand along first row
    for (int j = 0; j < n; j++) {

        // build the minor matrix (size (n-1) x (n-1))
        int minorMat[(n-1) * (n-1)];
        int r = 0, c = 0;

        for (int x = 1; x < n; x++) {   // skip row 0
            c = 0;
            for (int y = 0; y < n; y++) {
                if (y == j) continue;    // skip column j
                minorMat[r * (n-1) + c] = *(mat + x * n + y);
                c++;
            }
            r++;
        }

        // sign * element * determinant of minor
        int sign = (j % 2 == 0) ? 1 : -1;
        det += sign * *(mat + j) * determinant(minorMat, n - 1);
    }

    return det;
}

```

```c

// inverse of a matrix
// general interpretation is A inverse = 1/|A| * Adjoint(A), only 2x2 and 3x3
// matrices are handled


// calculate minor element of the matrix
static int minor(int i, int j, int *mat, int n, int rows, int cols) {
    int minorMat[(rows - 1) * (cols - 1)];
    int r = 0, c = 0;

    for (int x = 0; x < rows; x++) {
        if (x == i)
            continue;
        c = 0;
        for (int y = 0; y < cols; y++) {
            if (y == j)
                continue;
            minorMat[r * (cols - 1) + c] = *(mat + x * n + y);
            c++;
        }
        r++;
    }
    return determinant(minorMat, rows - 1);
}


// calculate co-factor
static int cofactor(int i, int j, int minorElement) {
    int sign = ((i + j) % 2 == 0) ? 1 : -1;
    return sign * minorElement;
}


// calculate adjoint
static void adjoint(int *mat, int n, int r, int c, float *adjOut) {
    for (int i = 0; i < r; i++) {
        for (int j = 0; j < c; j++) {
            int minorElement = minor(i, j, mat, n, r, c);
            int cofact = cofactor(i, j, minorElement);
            *(adjOut + j * n + i) = cofact; // transpose
        }
    }
}

// inverse calculator
void inverseMatrix(int r, int c, int *mat, float *inverseMat) {
    int detMat = determinant(mat, r);

    adjoint(mat, r, r, c, inverseMat);

    printf("Adjoint:\n");
    displayMatrixFloat(inverseMat, r, c);

```

3/16/26, 8:13 PM matrix.c

```
    for (int i = 0; i < r; i++)
        for (int j = 0; j < c; j++)
            *(inverseMat + i * c + j) /= detMat;
}
```

6.6 utils.c — Menu, Input & Output Utilities

code/utils.c

```c
#include "utils.h"
#include <stdio.h>
#include <stdlib.h>

// clear the screen
void clearScreen() {
#ifdef _WIN32
    system("cls");
#else
    system("clear");
#endif
}

// print menu for the home screen
void printMenu() {
    printf("\n╔═══════════════════════════╗\n");
    printf("│     MATRIX CALCULATOR     │\n");
    printf("╟───────────────────────────╢\n");
    printf("│  1. Addition              │\n");
    printf("│  2. Subtraction           │\n");
    printf("│  3. Multiplication        │\n");
    printf("│  4. Transpose             │\n");
    printf("│  5. Determinant           │\n");
    printf("│  6. Inverse               │\n");
    printf("│  0. Exit                  │\n");
    printf("╚═══════════════════════════╝\n");
    printf("Enter choice: ");
}

// inputting prompt for entries of the matrix
void inputMatrixPrompt(int *mat, int *r, int *c, char name) {
    printf("\n┌─ Matrix %c ──────────────┐\n", name);

    printf("│ Rows: ");
    scanf("%d", r);

    printf("│ Cols: ");
    scanf("%d", c);

    printf("│\n");
    printf("│ Enter elements row by row:\n");
    inputMatrix(mat, *r, *c);

    printf("└───────────────────────────┘\n");
}
```

7. Challenges

Several technical and design challenges were encountered during development:

7.1 Memory Management

The user determines matrix dimensions during program execution which causes 2D arrays to expand according to their input values. C programming language uses variable-length arrays (VLAs) to handle this requirement because their sizes become known during program execution instead of at compile time. The system required careful management of array sizes between functions because improper size transfer would result in out-of-bounds access during multiplication and inverse calculation operations.

7.2 Flat Array Indexing

You need to provide both row count and column count to every function when using 2D arrays because C language does not transmit array size data automatically. The function must receive exact dimensions of the matrix because it needs these details to process the matrix correctly through its complete range. The process of maintaining dimensional information throughout three multiplication operations and three cofactor expansion operations demanded continuous monitoring to prevent off-by-one errors in nested loop structures.

7.3 Recursive Determinant Stack Depth

For very large matrices, the recursive cofactor expansion creates deeply nested recursive calls, one per row/column reduction. For an n×n matrix, the recursion depth reaches n, and at each level the algorithm makes n recursive calls. This makes the algorithm $O(n!)$ in time complexity, which limits practical use to matrices smaller than approximately 10×10 before performance degrades significantly.

7.4 Floating-Point Precision

The singularity check for matrix inversion uses a threshold of `1e-9` (`if |det| < 1e-9, treat as zero`) because exact equality checks against 0.0 fail for matrices whose determinant is theoretically zero but computed as a very small non-zero floating-point value due to rounding during cofactor summation.

7.5 Multi-File Compilation

Coordinating function declarations in header files with definitions in source files, and ensuring that `runner.sh` includes all necessary source files in the GCC invocation, required understanding of the C compilation model (preprocessing, compilation, linking). Forgetting to include `matrix.c` or `utils.c` in the compile command results in undefined reference linker errors that can be confusing for beginners.

8. Conclusion

This project successfully implements a fully functional Matrix Calculator in C, demonstrating proficiency in core programming concepts including dynamic memory allocation, recursive algorithms, modular programming, and multi-file compilation. The calculator supports six fundamental matrix operations — addition, subtraction, multiplication, transpose, determinant, and inverse — through an interactive text-based menu.

The project reinforced the importance of careful memory management in C, the practicality of flat array representations for dynamic 2D data, and the elegance of recursive algorithms for mathematical problems such as cofactor expansion. The modular design separating computation (`matrix.c`), input/output (`utils.c`), and control flow (`main.c`) produces clean, maintainable code that is easy to extend.

Future improvements could include implementation of LU decomposition for more efficient determinant and inverse computation, support for non-square system solving (Gaussian elimination), and a graphical user interface. The project provides a solid foundation for further exploration of numerical methods and scientific computing in C.

9. References & Project Link

9.1 References

- Kernighan, B. W., & Ritchie, D. M. (1988). The C Programming Language (2nd ed.). Prentice Hall.
- Press, W. H., Teukolsky, S. A., Vetterling, W. T., & Flannery, B. P. (1992). Numerical Recipes in C: The Art of Scientific Computing. Cambridge University Press.
- Strang, G. (2016). Introduction to Linear Algebra (5th ed.). Wellesley-Cambridge Press.
- ISO/IEC 9899:2011 — C11 Standard for the C Programming Language.
- GCC Online Documentation: https://gcc.gnu.org/onlinedocs/

9.2 Project Link

GitHub Repository: https://github.com/AbdulRehman2896/Matrix-Calculator-

9.3 Build and Developed By

- Muhammad Abdullah Elahi — TC-25061 | Department of Telecommunications Engineering, NEDUET
- Abdul Rehman — TC-25065 | Department of Telecommunications Engineering, NEDUET

End of Report